

NLP Course Project

HUST

ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

ONE LOVE. ONE FUTURE.



ĐẠI HỌC
BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY
OF SCIENCE AND TECHNOLOGY

MULTITASK BERT FOR STUDENT SUPPORT UNDERSTANDING

Sentiment Analysis, Paraphrase Detection, and Semantic
Textual Similarity

Người thực hiện:

Vu Thuong Tin

Chu Anh Duc

Tran Quang Hung

Nguyen Xuan Khai

Do Dang Vu

ONE LOVE. ONE FUTURE.

School of Information and Communication Technology – HUST

1. Problem
2. Model
3. Training
4. Results
5. Demo
6. Conclusion

1. Project Goal

Goal: build a self-implemented multitask BERT system for three related NLP tasks.

- ▶ **SST:** five-class sentiment classification.
- ▶ **Quora Question Pairs:** binary paraphrase detection.
- ▶ **STS-B:** semantic textual similarity regression.

The same checkpoint is also used in a student-support demo: detect sentiment, identify duplicate questions, and retrieve relevant handbook entries.

2. Why Multitask Learning?

Training separately

- ▶ Each task learns its own representation.
- ▶ STS has less data and is harder to stabilize.
- ▶ Similar sentence-pair signals are not shared.

Our approach

Use one shared BERT encoder with task-specific heads. Paraphrase and STS additionally share a relational layer because both compare sentence pairs.

3. BERT Architecture

Core blocks

- ▶ Token ids
- ▶ Token embeddings
- ▶ Position embeddings
- ▶ Segment embeddings
- ▶ Self-attention
- ▶ Feed-forward
- ▶ Residual + LayerNorm

Diagram placeholder

- ▶ Insert BERT diagram
- ▶ Input → embeddings
- ▶ Encoder stack
- ▶ [CLS] output

4. System Architecture

Component	Role	Main file
Self-implemented BERT	Shared encoder	<code>bert_encoder.py</code>
SST head	5-class classifier	<code>heads.py</code>
Paraphrase head	Binary classifier	<code>heads.py</code>
STS head	Similarity regressor	<code>heads.py</code>
Relational layer	Shared Para–STS features	<code>relational.py</code>
Trainer	Interleaved multitask training	<code>training/</code>

The encoder forward pass does not call `transformers.AutoModel`. It implements embeddings, attention, feed-forward layers, residual paths, layer normalization, and the pooler directly.

5. Rich Relational Layer

Paraphrase and STS both compare sentence pairs, so they share an intermediate interaction layer.

$$z = \frac{h_u \odot h_v}{\|h_u\|_2 \|h_v\|_2 + \epsilon}$$

- ▶ h_u, h_v : pooled BERT vectors for two sentences.
- ▶ Element-wise interaction keeps more information than scalar cosine similarity.
- ▶ The shared layer helps STS benefit from the larger Quora paraphrase dataset.

6. Training Pipeline

Steps

- ▶ Load tokenizer
- ▶ Load BERT weights
- ▶ Build dataloaders
- ▶ Interleaved batches
- ▶ Task losses
- ▶ Save best checkpoint

Diagram placeholder

- ▶ Insert training flow
- ▶ SST / Para / STS
- ▶ Shared encoder
- ▶ Task heads
- ▶ Aggregate score

Best recipe: AdamW, LR $2e^{-5}$, 20 epochs, linear warmup/decay.

7. Self-Implementation Check

We re-coded BERT

Full encoder implemented from scratch:

- ▶ No `AutoModel` forward pass.
- ▶ Max hidden-state diff: 4.29×10^{-6} .
- ▶ Self-impl score: **0.759**.
- ▶ HF baseline score: 0.761.

The custom encoder is numerically faithful and nearly matches the library-backed model.

8. Evaluation Metric

Each task has its own metric:

- ▶ SST: accuracy.
- ▶ Paraphrase: accuracy.
- ▶ STS: Pearson correlation.

Aggregate score:

$$S = \frac{SST_{acc} + Para_{acc} + (STS_r + 1)/2}{3}$$

STS Pearson correlation is normalized from $[-1, 1]$ to $[0, 1]$ before averaging.

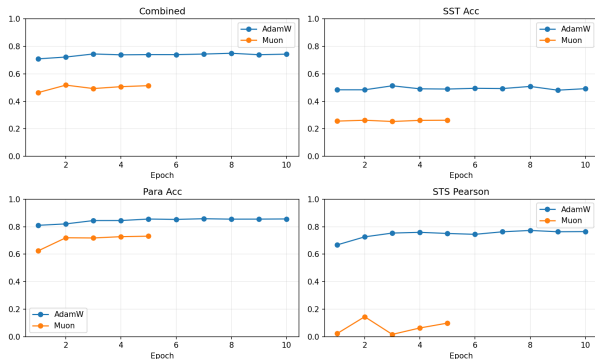
9. Main Results

Model	SST	Para	STS	Score
Self-impl AdamW linear 20e	0.520	0.864	0.785	0.759
HF sanity baseline 10e	0.526	0.861	0.790	0.761
Self-impl AdamW constant LR	0.505	0.860	0.784	0.752
Self-impl Muon safe 20e	0.523	0.851	0.750	0.750
Muon high-LR early run	0.262	0.720	0.145	0.518

AdamW with linear warmup/decay is the best self-implemented setup. The HF backend is only a sanity baseline, not the main submission path.

10. AdamW Beats Muon

Report-best AdamW vs Muon training history



Decision

- ▶ AdamW is clearly better.
- ▶ Muon damages pretrained weights.
- ▶ Para / STS drop sharply.
- ▶ Final choice: AdamW.

11. Demo A: Checkpoint Inference

Purpose

Load one checkpoint and test the three benchmark modes directly:

- ▶ SST sentiment prediction.
- ▶ Paraphrase probability.
- ▶ STS similarity score.

`scripts/demo_gradio.py`

Screenshot Placeholder

Replace with the Gradio checkpoint inference screen.

12. Demo B: Smart Student Support

Product workflow

- ▶ Detect student sentiment or urgency.
- ▶ Check whether a new question duplicates an FAQ.
- ▶ Rank HUST handbook entries by semantic similarity.

`scripts/demo_student_support.py`

Screenshot Placeholder

Replace with the Smart Student Support Assistant screen.

13. Demo Commands

Checkpoint inference demo

```
PYTHONPATH=src python -m scripts.demo_gradio \  
-config configs/report_best_self_impl_20e_linear_warmup_lr2e5.yaml \  
-checkpoint runs/report_best_self_impl_20e_linear_warmup_lr2e5/best.pt
```

Smart student support demo

```
PYTHONPATH=src python -m scripts.demo_student_support \  
-config configs/report_best_self_impl_20e_linear_warmup_lr2e5.yaml \  
-checkpoint runs/report_best_self_impl_20e_linear_warmup_lr2e5/best.pt
```


14. Conclusion

- ▶ We re-coded the full BERT encoder.
- ▶ Max hidden-state diff: 4.29×10^{-6} .
- ▶ Self-impl score: **0.759**.
- ▶ HF baseline: 0.761.
- ▶ AdamW clearly beats Muon.
- ▶ Muon damages pretrained weights.
- ▶ Two demos show both benchmark inference and a concrete student-support use case.

Future work: better STS calibration, dynamic task weighting, and stronger relational sentence-pair features.

THANK YOU

Project: Multitask BERT for Student Support Understanding

Repository: multitask-bert-v2

Q&A

The logo for HUST (Ho Chi Minh University of Science) is displayed in white, bold, sans-serif capital letters. It is centered within a dark blue rectangular area. Surrounding the text is a decorative pattern of red dots of varying sizes, arranged in concentric, slightly irregular circles, creating a textured, circular effect.

- [1] Devlin et al., *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*, 2019.
- [2] Liu et al., *Multi-Task Deep Neural Networks for Natural Language Understanding*, 2019.
- [3] Reimers and Gurevych, *Sentence-BERT*, 2019.
- [4] Loshchilov and Hutter, *Decoupled Weight Decay Regularization*, 2019.